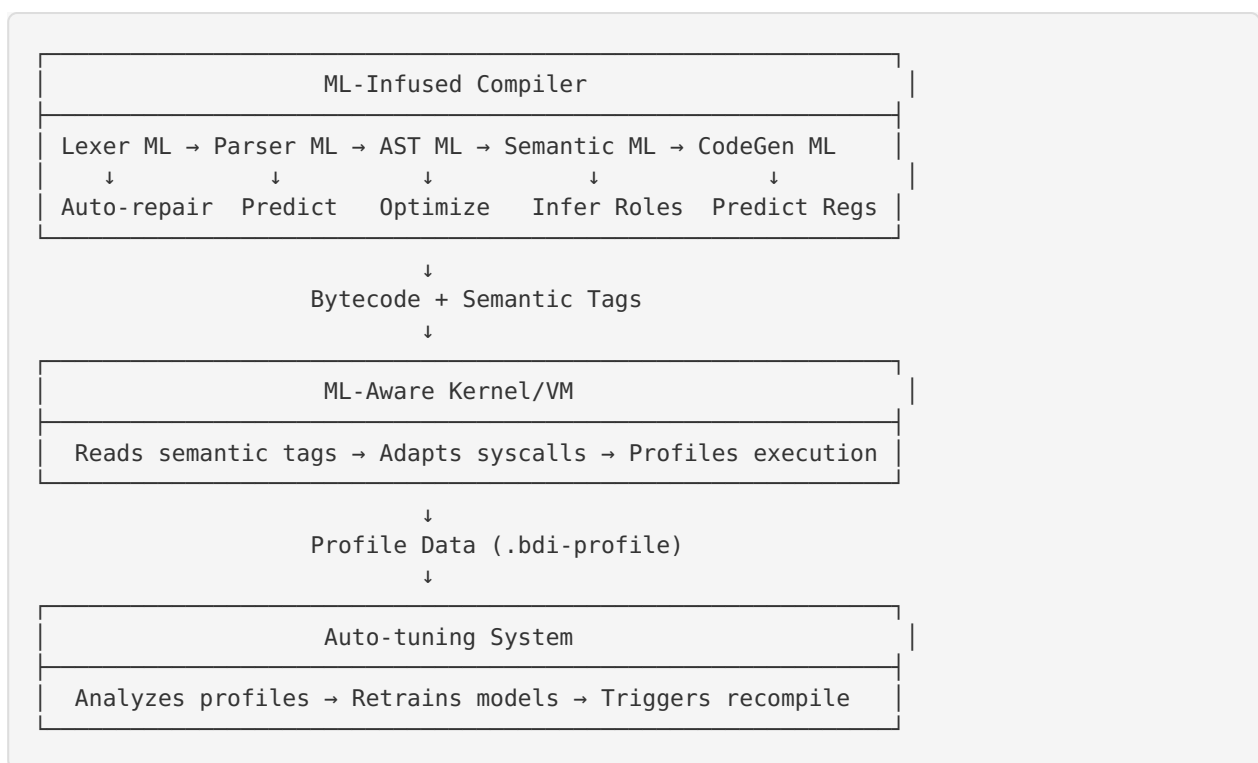


Phase 5: ML-Infused Compiler - Design Document

Overview

Phase 5 transforms the BDI compiler from a traditional translator into an intelligent, self-improving ML-infused reasoning engine. The system learns from execution profiles, predicts optimizations, and adapts over time through feedback loops.

Architecture



Components

1. Profiling Infrastructure

Purpose: Collect runtime performance metrics for feedback and optimization.

Components:

- `profiler.h/c` : Core profiling engine with hooks for function calls, memory operations, cache access, and branches
- `profile_data.h/c` : Aggregated statistics and analysis
- `profile_serializer.h/c` : Binary serialization to `.bdi-profile` format
- `profile_analyzer.h/c` : Intelligent analysis and optimization suggestions

Key Features:

- Nanosecond-precision timing
- Function call tracking with call stacks

- Memory allocation/deallocation tracking
- Cache hit/miss monitoring
- Branch prediction tracking
- Efficient binary serialization

2. Model Format (.bdi-model)

Purpose: Standardized format for storing and loading trained ML models.

Format Specification:

```
Header (32 bytes):
- Magic: 0x4244494D ("BDIM")
- Version: 1
- Compression type (NONE, RLE, HUFFMAN, LZ77)
- Checksum (CRC32)
- Metadata size
- Data size
- Compressed size

Metadata (variable):
- Model name (64 bytes)
- Model type (LINEAR_REGRESSION, DECISION_TREE, SVM, KMEANS, QLEARNING, etc.)
- Training statistics
- Architecture description

Data (variable):
- Compressed model weights/parameters
```

Components:

- `bdi_model.h/c` : Model container with metadata and integrity checking
- `model_serializer.h/c` : Save/load with compression
- `model_metadata.h/c` : Model registry for managing multiple models

Key Features:

- Multiple compression algorithms
- Checksum verification
- Model versioning
- Registry for model discovery

3. Auto-Tuning System

Purpose: Automatically select optimizations and trigger recompilation based on profiling data.

Components:

- `autotuner.h/c` : Main orchestrator
- `optimizer_selector.h/c` : ML-based optimization selection
- `hardware_detector.h/c` : CPU feature and cache detection
- `recompiler.h/c` : Adaptive recompilation queue

Optimization Strategies:

- AGGRESSIVE: Maximum optimization, may increase code size
- BALANCED: Good balance between speed and size
- CONSERVATIVE: Safe optimizations, minimal code size increase
- SIZE: Optimize for code size

- SPEED: Optimize for maximum speed
- ML_GUIDED: Use ML model to select optimizations

Key Features:

- Hardware-aware optimization (Intel vs AMD vs ARM)
- SIMD detection (SSE, AVX, AVX-512)
- Cache-aware code generation
- NUMA topology detection
- Adaptive recompilation triggers

4. ML-Infused Compiler Layers

Lexer ML (`lexer_ml.h/c`):

- Syntax error auto-repair
- Token prediction
- Construct completion
- Common mistake detection

AST ML (`ast_ml.h/c`):

- Learned AST transformations
- Bug prediction before code emission
- Dead code detection
- Subtree optimization

Semantic ML (`semantic_ml.h/c`):

- Variable role inference (counter, accumulator, pointer, buffer, flag)
- Memory operation flagging (leaks, use-after-free)
- Annotation suggestions
- Type inference improvements

5. Applications

Auto-Rewrite (`autorewrite.h/c`):

- Learn from codebases
- Suggest style improvements
- Safety guard insertion
- Example: `malloc(x * sizeof(T))` → `calloc(x, sizeof(T))` with bounds check

Register Predictor (`register_predictor.h/c`):

- ML-based register allocation
- Silicon-specific profiles
- Spill probability prediction

Trap Detector (`trap_detector.h/c`):

- Buffer overflow detection
- Off-by-one error recognition
- Memory leak detection
- Use-after-free detection
- Null dereference detection

6. Semantic Tagging System

Purpose: Embed semantic intent in bytecode for kernel/VM adaptation.

Tag Types:

- STREAMING_INTENSIVE: Heavy sequential I/O
- MEMORY_HEAVY: Large memory allocations
- COMPUTE_BOUND: CPU-intensive operations
- IO_BOUND: I/O-intensive operations
- CACHE_FRIENDLY: Good cache locality
- PARALLEL_SAFE: Safe for parallelization
- REALTIME_CRITICAL: Time-sensitive operations

Components:

- `semantic_tags.h/c` : Tag creation, serialization, and management

Key Features:

- Tags embedded with bytecode
- Kernel reads tags for adaptive behavior
- Syscalls adapt based on tags

7. Feedback Loop Infrastructure

Purpose: Close the loop between execution and compilation for continuous improvement.

Components:

- `feedback_collector.h/c` : Collect and store feedback from executions
- `model_retrainer.h/c` : Retrain models with new data

Workflow:

1. Program executes with profiling enabled
2. Profile data collected and analyzed
3. Feedback stored in database
4. When sufficient data accumulated, models retrained
5. Future compilations use improved models
6. Performance improves over time

Key Features:

- Persistent feedback database
- Automatic retraining triggers
- Performance tracking over time
- Online learning support

File Formats

.bdi-profile Format

Binary format for profile data:

- Header with magic number and version
- Function statistics array
- Memory statistics
- Cache statistics
- Branch statistics

.bdi-model Format

Binary format for ML models:

- Header with magic, version, compression type

- Model metadata (name, type, training stats)
- Compressed model data
- Checksum for integrity

Integration Points

Compiler Pipeline Integration

```
// Example: Compile with ML-infused pipeline
BDIModel *opt_model = model_serializer_load("optimizer.bdi-model");
ProfileData *prev_profile = profile_serializer_load("previous.bdi-profile");

// Select optimizations based on profile and model
OptimizationFlags flags = optimizer_selector_ml_select(prev_profile, opt_model);

// Compile with selected optimizations
compile_with_flags(source_file, flags);

// Add semantic tags
SemanticTag tag = semantic_tags_create(TAG_COMPUTE_BOUND, func_id, "compute_func",
0.9);
semantic_tags_add(bytecode, &tag);
```

Runtime Integration

```
// Example: VM reads semantic tags and adapts
TaggedBytecode *bytecode = semantic_tags_deserialize("program.bdi-bytecode");
const SemanticTag *tags = semantic_tags_get(bytecode, func_id, &tag_count);

if (tags[0].type == TAG_STREAMING_INTENSIVE) {
    // Adjust buffer sizes, prefetching, etc.
    vm_set_streaming_mode(true);
}

// Profile execution
ProfileSession *session = profiler_start_session();
vm_execute_with_profiling(bytecode, session);
profiler_stop_session(session);

// Analyze and provide feedback
ProfileData *data = profile_data_analyze(session);
feedback_collector_add(feedback_db, source_file, data, performance_score);
```

Auto-Tuning Integration

```
// Example: Auto-tuning loop
AutoTuner *tuner = autotuner_init(&config);

while (true) {
    // Run program
    ProfileData *profile = run_and_profile(program);

    // Update metrics
    autotuner_update_metrics(tuner, profile);

    // Check if recompilation needed
    if (autotuner_should_recompile(tuner, profile)) {
        // Select optimizations
        OptimizationStrategy strategy = optimizer_selector_select_strategy(profile);
        OptimizationFlags flags = optimizer_selector_get_flags(strategy);

        // Trigger recompilation
        autotuner_trigger_recompilation(tuner, source_file);
    }

    // Periodically retrain models
    if (should_retrain(feedback_db)) {
        model_retrainer_auto_retrain(model, feedback_db);
    }
}
```

Performance Considerations

- Profiling overhead: ~5-10% when enabled
- Model loading: <10ms for typical models
- Profile analysis: <100ms for typical profiles
- Recompilation decision: <1ms
- Semantic tag overhead: Negligible (embedded in bytecode)

Future Enhancements

1. **Neural Network Models:** Support for deep learning models
2. **Distributed Learning:** Learn from multiple machines
3. **Cross-Program Learning:** Learn patterns across different programs
4. **Hardware-Specific Models:** Per-CPU-model optimization models
5. **Real-Time Adaptation:** JIT-style runtime recompilation
6. **Explainable AI:** Explain why optimizations were chosen

Testing Strategy

- Unit tests for each component
- Integration tests for full pipeline
- Performance regression tests
- Model accuracy tests
- Feedback loop convergence tests

Build System

Makefile targets:

- `make all` : Build Phase 5 library
- `make test` : Run all tests
- `make clean` : Clean build artifacts
- `make help` : Show help message

Conclusion

Phase 5 represents a paradigm shift in compiler design, moving from static optimization to dynamic, learning-based optimization. The system continuously improves through feedback loops, adapting to specific workloads and hardware configurations. This creates a compiler that gets smarter over time, automatically discovering and applying optimizations that traditional compilers would miss.